

《3D Vibe Coding》橙皮书

386 页开源手册盘点 + 三章深挖：一个人用 Codex × Tripo 造出一座能开车逛的巴黎

386 页 / 25.6 万字 / 260 图

6 个可玩 demo

56 件资产逐件对账

CC BY-NC-SA 4.0

花叔 @alchaincyf

v260914c

269★

这本书是什么：花叔做“能开车逛的巴黎”时的全程实测记录——不会写 3D 代码的人，用 AI 生成 3D 资产（Tripo 图生 3D），再把它们真正接进能跑的项目（Three.js / Blender / Unity）。**最大特色不是“教会你”，而是“每一条结论都有能核查的数”：**价格是坐在表单前逐个开关试出来的，面数是解析 GLB 索引数组数出来的，翻车的地方（927 万 vs 927K、65 积分废件）原文留着没删。

 仓库：github.com/alchaincyf/3d-vibe-coding-handbook (book/ 全书 HTML+339 图, demos/ 三个 demo 源码, scripts/ 11 个工具脚本, manifest/ 56 件资产清单)。资源包 11 个共 229MB 按 \$00 索引下载，不出整包。

盘点

全书目录：\$00-\$22 + 附录 A-D

章节	一句话内容
§00	这本书怎么用 + 11 个资源包索引 (按需下载, 各带 SHA256)
§01 深挖	十分钟跑通第一次图生 3D: 两种模式、价格结构、面数框的口径陷阱
§02	参考图怎么准备——"图生 3D 的成败, 八成不在开关, 在你喂进去的那张图"
§03	生成之后: 导出弹层、API 直链绕开"另存为"弹窗、签名链接约两天过期
§04 深挖	减面与重拓扑: E10 对照实验、1.75 换算系数、减面下限、"卡顿先量贴图"
§05	智能拆分与贴图生成 (拆件不吃四边面、不吃已绑骨模型)
§06	绑骨与动画: 自动绑骨 20 积分、三模式对照
§07	怎么把积分花在刀刃上: 每个动作的基础价与加项逐笔对账
§08	为什么先出设计图再建模 (设计图 = 可复现的中间产物)
§09	提示词技巧示例 (含图)
§10	批量出图与命名
§11 深挖	把 Tripo 接进你的工具: CLI / Codex 插件 / MCP / API / DCC Bridge 五条路
§12 深挖	Three.js: 环境贴图 / 贴图显存 / 骨骼复制三坑 + 性能测量方法论
§13	Blender: 什么时候该进 (绑骨权重), 什么时候别进 (减面等价)
§14	Unity: 把资产搭成会动的园区
§15	谁负责哪一层: 近景 AI 模型 / 远景代码渲染的分工判据
§16	从选题到资产清单 ("23、25、29 数的不是同一批东西"的对账表)
§17 深挖	生成与清洗流水线: 逐件生成 → 导出 GLB → 批量减面 → 归一化尺寸
§18	组装: 把 23 件模型变成一个能开的城 (实例配额、A/B 帧时间)
§19	验收与发布
§20	机构类案例: 一台能拆能装的四冲程发动机 (曲柄滑块按真实运动学算)
§21	艺术类案例: 星月夜拆五层, 两层换 Tripo 真 3D
§22	世界类案例: 小王子星球 (28 米直径走得完)
附录A	56 件资产逐件规格 (三角面/顶点/贴图/积分, 可当验收单, 以文件读数为准)
附录B	踩坑索引: 按症状查的五张表——"已经动手了卡在某个症状上"先翻这里
附录C	配置与积分参考
附录D	术语人话版 + 面数换算的完整分布

读法建议 (本书自己的路由表): 没跑过图生3D → §01; 模型生出来了不知道下一步 → §03; 卡在具体症状 → 附录B; 本指南深挖 §01/§04/§12 三章 (与已学的 TripoSR、glTF 管线、Three.js 衔接最紧), §11/§17 简挖。

Q1 图生 3D 是什么？和我学过的 TripoSR 什么关系？

一句话：上传一张设计图，几十秒后得到一个能拖着转、能下载的 3D 模型（GLB）。背后的学术底座就是 vault 里已学的 [[LRM大型重建模型]] 前馈范式——TripoSR 是 Tripo (VAST) 的开源研究版（2 秒本地出灰模），Tripo Studio 是同一家**商业产品管线**：灰模只是起点，后面还挂着贴图生成、智能拆分、重拓扑、自动绑骨一整条服务链。

🔗 互链：[[LRM大型重建模型]]（一次前向出 3D 的原理）、[[Triplane三平面表示]]（内部表示）、[[glTF与GLB资产管线]]（产物格式）、项目笔记 [[项目笔记/triposr]]（开源版实测）。



§17 四步流水线；真正吃时间的是第一步排队——所以“并行提交、串行清洗”

图1：从一句话到能跑的场景——本书的整条流水线（类比 [[混合模型分工流水线]]：设计流出 AI，机制流出代码）

这条流水线的分工哲学 (§15)：近景要细节的交给 AI 模型，远景只要“那儿有个东西”的交给代码——巴黎城里 2 万株行道树是程序化的（9 个绘制调用/8.4 万三角面），只有近景 110 株用 Tripo 的 GLB。

Q2 "Generate 50"这个数字是怎么算出来的？

花叔干了件笨事：坐在表单前把"几何与贴图"弹层里的开关一个一个动，看按钮上的数怎么跳（全程只看不按，0 积分）。跳出来的结构特别规整：

状态	按钮	拆解
只关掉 Texture（灰模）	⚡ 15	几何底价 = 15
贴图 2K	⚡ 30	+15
贴图 4K	⚡ 40	+25
贴图 8K（默认！）	⚡ 50	+35 ← 默认价的一半在买 8K 贴图
8K 再开 Ultra Mesh	⚡ 65	Ultra 另算 +15（不在默认 50 里）
智能网格 P2.0	⚡ 65 + 30	网格 65（原价 100 划线），贴图另付 30

默认 50 分里 35 分买的是 8K 贴图——而做网页/游戏导出时本来就要降到 1k。生成选 2K、导出选 1k，两头都省：一件 30 vs 一件 65，×40 件差 1400 积分，接近半个月订阅额度（专业版 ¥140/月 3000 积分，免费档 200 积分约 13 个模型）。

三个账单陷阱：① Ultra 那 15 分**不在**默认 50 里，算进去预算凭空多四分之一；② "自动保存您的设置"是跨会话的——为特写件开的档会一直跟着后面每一件；③ 首页快捷框和工作台默认值不同（55 vs 50），规矩只有一条：**以按下去之前按钮上当场显示的数为准。**

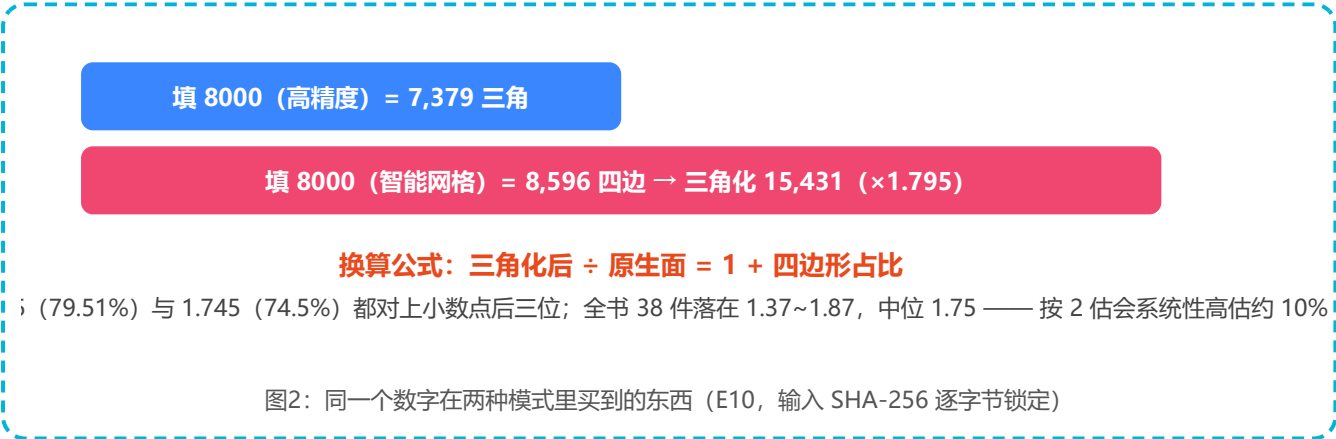
Q3

面数框骗局：同一个 8000 为什么不是同一个 8000？

两种模式的"面数"框数的**不是同一种面**：高精度模型（v3.1）出三角面，那个数就是三角面；智能网格（P2.0）出四边面，那个数数的是四边面。引擎最终只认三角面——四边面进引擎要拆成两个。E10 实验（同一张出租车图、都填 8000、只换模式，输入图 SHA-256 三方对齐）：

	A 高精度 v3.1 (15 分)	B 智能网格 P2.0 (65 分)
原生面	7,379 三角面	8,596 四边面 (占 79.51%)
三角化后	7,379	15,431
顶点	3,646	8,580

同一个 8000，智能网格拿到的几何预算是高精度的 2.09 倍——B 的保险杠栅条、大灯圈、门把手、雨刷全都在，A 同位置是几块糊过去的面。跨模式比面数先换算，不然结论会反。



面数框另外两个咬人处：它是目标不是硬上限（38 件里 35 件超出设定，中位超 13%）；"自动保存"管不了它——**每件都要重设**。凯旋门那件：浏览器自动化往框里打字 15000，界面明明白白写着 15000，接口读回 face_limit=5000（默认值），65 积分打水漂。根因是响应式框架托管的输入框：直接改 DOM 的 value，框架内部状态收不到通知。正确写法是调原型 setter 再补事件，硬验证是设完按一下 → 方向键：变 15002 说明真进去了，跳回 5002 就是白设。

Q4 减面还是重拓扑？能减到多少由什么决定？

减面 (decimation) 在原来的面上合并删减，拓扑变乱；**重拓扑 (retopology)** 是同一形状重新铺一张规整的网 (10 积分，要进 Blender 改/绑骨才需要)。先说反直觉的事实：巴黎 29 件资产面数 16,711~46,196、中位 18,509，没有一个超 10 万——整个项目一次重拓扑都没用上。所以第一个动作不是打开重拓扑页，是先数面数。

减面的下限由几何有多光滑决定：减面算法找相邻顶点判断"合并了看不看得出来"。连续曲面（人体）砍到 3.3% 还认得出是个人；建筑全是直角窗洞只能到 37~44%；铁塔的格构结构和树的分离薄片几乎砍不动 (72.5%~95.3%)。光滑度 = 有多少个可以被牺牲的顶点。所以别拿一套统一参数跑全库：人物/载具/道具 0.25、建筑 0.45、树和铁塔 0.75。

情况	动作	成本
面数已在 1.5~3 万，做静态道具	什么都不用做	0
几十万面（镂空建筑）/ 要绑骨	重拓扑（四边面）	10 分
一批资产整体瘦身进网页	gltf-transform 按类型分配比例	0 分，5.2 秒/个
加载慢、显存爆、风扇狂转	先量贴图 ，多半是 resize 不是 simplify	0 分
形状本身就不对	改参考图重新生成	30~50 分

✅ **实验复现（见本指南实验一）：**光滑球面 720 三角砍到 35 (4.9%) 触底；格构塔 1152 → 336 (29.2%) 触底，ratio 0.1/0.05/0.02 输出完全相同——"再减就崩"的硬下限现象亲手复现。

Q5 卡顿的第一反应为什么应该是查贴图而不是减面？

小王子 demo 卡得没法看，本能反应是"减面、关泛光、砍阴影"。先量了一次：**整条渲染链 GPU 只吃 0.37ms，预算 (16.7ms@60fps) 的八分之一都没用完——三角形是闲的。真凶在显存：**

一张 8192² 贴图的显存账

RGBA32 展开 = 268 MB + mipmap 链 ×4/3 ≈ 358 MB / 张

17 件资产 31 张贴图 (10×8K + 20×4K) = 5.0 GiB —— 卡到没法看 →

降到 1024²: 0.22 GiB (23×) , 三角面 147,879 一个没动, 肉眼看不出差别

图3: 显存才是瓶颈——只跑 `gltf-transform resize --width 1024` , 几何一个面不动

指标	改前	改后	倍数
贴图显存	5.0 GiB	0.22 GiB	23×
GLB 载荷	89.87 MiB	12.30 MiB	7.31×
资产就位	3.90 s	0.70 s	5.6×
JS 每帧 (中位)	1.5 ms	0.7 ms	2.1×
三角面	147,879 —— 未动		

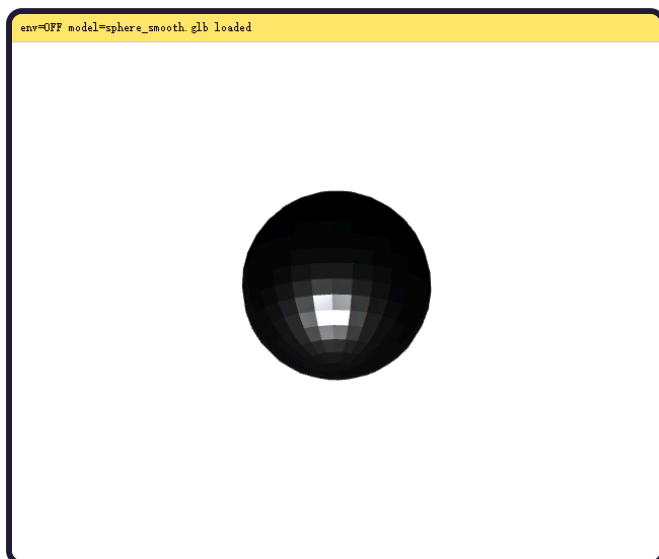
为什么显存这么爆：每个道具在画面里才几百像素，却挂着一张 8192² 底图，GPU 一直在做纹理换入换出，表现出来就是"卡"。用 `resize` 别用 `optimize`——optimize 默认顺手跑 simplify，而几何根本不是瓶颈。顺序：**先量 GPU 耗时和绘制调用 → 量不出问题先砍贴图 → 再合并绘制调用 → 最后才轮到砍面。**

按绘制调用分档比按三角面分档有用：巴黎 244 万三角 / 77 个绘制调用，渲染一帧 5.0ms，离预算还有三倍余量；"全场景 30 万面"那条流行经验值被实测推翻——照它做会把塞得下的东西砍掉。

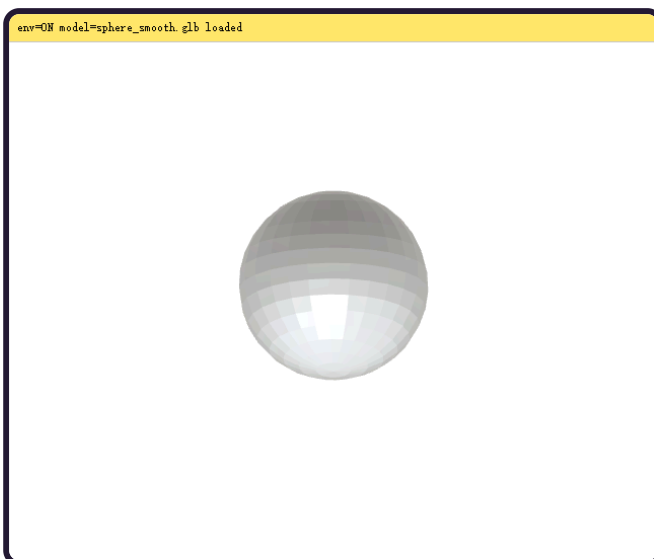
Q6 PBR 金属在白底上为什么死黑？（已亲手复现）

发动机 demo v1 深色底一切正常，v2 改纸感白底后整台机器变成一团黑影。原理：金属度高时漫反射被抑制，只靠直接光什么也看不见；深色背景把问题吃掉了，白底一换全暴露。**模型没坏——缺的是环境贴图**

(scene.environment)。两条路：要金属质感就做环境贴图；不要就把 metalness 摁到 0 (roughness 给足，让它以哑光身份好看)。最怕的是既不做环境贴图又保留高金属度。



✗ env=OFF: metalness=1 的球在白底上死黑，只剩直射光高光点（中心区平均亮度 72.9/255）



✓ env=ON: 书里的 studioEnv() canvas 渐变环境贴图接上后，正常的镀铬球（亮度 206.7, 2.8×

studioEnv() 两个试出来的细节：① 渐变不能全亮——必须从 #ffffff 压到 #403c35 给暗部（摄影棚的黑旗），金属才有明暗对比；② 白底场景别用 ACESFilmic 色调映射（会把白底压成灰）。不需要任何外部 HDR 文件，canvas 画渐变 → PMREMGenerator 转环境光照。

连带坑：vendor 精选 5 个文件里 BufferGeometryUtils 不是可选的（r160 的 GLTFLoader 自己会 import，缺了的表现是“页面全白”而非报缺文件）；P2.0 资产只出一张 base color（没有法线图和 ORM），别拿着“Tripo 出 PBR 三件套”去 P2.0 产物里找不存在的文件。

Q7 怎么把 Tripo 接进 Agent 工作流？五条路一个钱包

五条接入路其实是两类：**让 AI 替你调生成**（CLI / Codex 插件 / MCP——门槛是 Node 20+ 和一次浏览器登录）和**把模型搬进你的软件**（API / DCC Bridge）。三条 Agent 路都实测到"发起生成"才被拦下，拦的理由非常有教育意义：

Studio 订阅和 API 钱包是两套账，官方明说不互通。专业版订阅生成按钮照常能点，API Wallet 是 0/0 Credit——CLI、Codex 插件、MCP 三条路共享同一个 API 钱包，一分钱不通兑。要为 Agent 化使用单独充值。

路径	门槛	实测要点
Tripo CLI (npm -g tripo-cli)	低	doctor 未登录查 6 项/登录后 8 项；设备码登录 210 秒（页面会丢 query，验证码必须手打）； <code>tripo view x.glb</code> 白送一个零积分本地 3D 预览器
Codex 插件	最低	<code>codex plugin add tripo-3d@openai-curated-remote</code> 14 秒，不用 GUI；它不是 MCP server，是 两个 Agent Skill ，底层调同一个 CLI
MCP (CLI 自带 tripo mcp)	中	stdio server, 5 工具 (make/task_get/task_wait/balance/history)；配进 Claude Code: <code>claude mcp add --scope local tripo -- tripo mcp --quiet --no-open</code>
API (V3)	高	异步 task 轮询；官方 Game-Ready Character 管线 = image-to-model → rig-check → rig → retarget (约 105 秒/85 分)；V2 API 2026-11-01 下线
DCC Bridge	低 (要 Pro)	浏览器生成一键推进 Blender/Unity 等 10 个软件，纯传输通道不涉 AI

退出码表值得贴在脚本里（把"白花钱"和"没花钱"分开）：3 认证错误 / **4 积分不足**（没开始生成，不扣，实测印证） / 5 内容拒绝 / 6 任务失败（自动退款） / 9 限流退避重试。另：`quad=true` 强制 FBX 输出——四边面拓扑存不进 GLB。

Agent 不该替你做的五件事：浏览器授权页点 Authorize（设备码流程的安全根基）、充值/领 14 天试用钱包、保证浏览器登录态、额度耗尽的决策、sudo npm 密码。

Q8 P2.0 资产接入的五道工序与朝向陷阱

九月把沿街资产整批换成智能网格 P2.0 重出的件后发现：**这是另一条路，坑几乎不重合**。五道工序少一道就装不上去：

工序	为什么
① FBX → GLB	P2.0 带贴图只给 FBX（四边面存不进 GLB），浏览器不认；用 Blender headless 转（Creator 字段就是 Blender，同实现往返最稳）
② 降 8K 贴图	$8192^2 \times 26 \text{ 件} = 9.3 \text{ GB}$ ，纸上就能算出来的显存灾难
③ 定 yaw	朝向不能靠算（见下），每件渲六视图人工看
④ 烘变换	旋转缩放乘进顶点，不写节点矩阵——“量的时候算了矩阵、用的时候没算”的不对称会让模型静默错位
⑤ 量 A/B 帧时间	确认没把性能换坏（GPU timer query，见 Q11）

朝向三类资产三套惯例，混成一句话记一定会出事：绑骨人形与 +Z 差 90° (yaw=-90)；v3.1 静态行人朝 +Z（换 P2.0 时 -90）；v3.1 建筑朝 -Z（换 P2.0 时 +90，和行人反号而包围盒长得一模一样）；P2.0 家族 38 件里 36 件朝 +X，还有 2 个例外靠渲图读出来。包围盒推不出朝向（转 90° 只把 X/Z 互换，--yaw 90 和 --yaw -90 的包围盒完全相同），四视图分不出正反——必须渲六视图，在哪个视图里看见正面，正面就朝那个方向。

另一个“不像格式问题”的坑：KHR_mesh_quantization 的顶点是 normalized int16，自写解析器不认这个标志就把原始整数当坐标——1 米高的人读成 32767 单位，缩放系数算出 32783。一个只在一半输入上出错的 bug，比全错的难找得多。

Q9 为什么整排行人会“同手同脚”？

巴黎城 18 个行人里 6 个共用同一个绑骨模型。第一版用普通 `clone()` 复制——所有实例共享同一副骨骼，于是排队走正步。修法三件套：

```
import {clone as skeletonClone} from 'three/addons/utils/SkeletonUtils.js';
const root = skeletonClone(walkScene); // ① 骨骼独立复制
const mixer = new THREE.AnimationMixer(root); // ② 每实例一个 Mixer
mixer.clipAction(walkClip).play();
mixer.setTime((x*0.37 + z*0.11) % walkClip.duration); // ③ 相位按坐标错开，否则还是仪仗队
```

另外两条绑骨特有坑：① **切掉动画的位移轨**——retarget 出的剪辑里 Hip.position Y 位移范围 1.338，而人才 1 米高；行走该是纯旋转，位移归游戏代码管，否则"走着走着弹回一大步"（Tripo 侧对应导出时勾"原地播放动画"）。② 带骨骼的模型不能用 `Box3.setFromObject` 量尺寸（包围盒缓存不随姿势更新，还叠 Armature 变换）——绕开 Three.js 直接读 GLB 的 POSITION accessor min/max，repo 里的 `scripts/measure_glb.py` 就是这个工具（还处理了蒙皮矩阵）。

Q10 近景 AI + 远景程序化：这笔账怎么算？

巴黎城行道树：程序化实例画 2 万株只吃 9 个绘制调用 / 8.4 万三角面。想全换成 Tripo 的 GLB 树（15k 三角/株）？近景配额 $150 \text{ 株} \times 15\text{k} = 225 \text{ 万三角} = \text{整层预算的 26 倍}$ 。最终折中：近景 110 株 GLB（真实树冠+叶片贴图），中远景全程程序化。



图4：AI 资产的面数成本撑不住数量——判据是“近景要细节，远景只需要那儿有个绿色的东西”（\$15 分层）

接 GLB 树时还踩了**缩放基准**坑：城市层的 c.h 是“冠高”，GLB 树宽高比接近 1:1，按整树高度放大 2 倍时冠幅 14 米、而行道树间距才 8 米——整排糊成一堵绿墙。归一化前先问：**这个资产靠哪个维度和邻居对齐？**树按冠幅（heightScale 取 1.2）、建筑按临街面宽、人物按身高。还有一个一行代码的体检：顶点高度 p90 明显低于总高 70%，说明包围盒是被一根细杆/一顶伞撑起来的（取暖伞案例：桌椅从 1.1 米被压到 0.5 米），不能拿总高做归一基准。

Q11 量性能：帧间隔会骗人，GPU timer 不会

换完 6 件资产要回答"有没有把性能换坏", 第一次就量错了: 帧间隔被 vsync 锁在 8.3/16.6ms, 两组资产中位数一模一样, 差别全被锁没了。要量的是每帧 GPU 真正忙了多久——挂 WebGL2 扩展 `EXT_disjoint_timer_query_webgl2` 在 requestAnimationFrame 前后 beginQuery/endQuery, 隔几帧收结果 (GPU_DISJOINT 为真则丢弃)。三条协议纪律:

① 串行测量——另一轮和 headless 录屏抢同一块 GPU, 量出中位 8.57ms/p95 61.8ms 的离谱方差, 最小值和干净机器吻合判断是干扰, 但这是推断不是测量, 正确处理是丢掉重测; ② 固定采样协议 (刷新→自由漫游→回出生点静止采样); ③ A/B 表的结论是"量不出回归"而不是"变快了"——0.37ms 的差落在跑间噪声里, 没有第二次独立跑就不说变快。把噪声读成改进, 是性能测量里最常见的自欺, 尤其在你正好希望它变快的时候。

🔗 这套"先量后动、噪声不当改进"与 vault 里 [[证据优先质检ProofOverClaims]]、[[事实与判断分离]] 是同一个家族: 测量本身也要被怀疑 (自测脚本的两个 bug 是同一轮清出来的)。

Q12 这本书最值钱的可能不是 3D, 是验证文化

全书处处是方法论级的自觉, 随便列五条 (都可平移到任何工程领域):

做法	原话/事实
SHA-256 锁输入	"我记得传的是同一张图"和"两个文件 SHA-256 逐位相同", 证据等级差一整个数量级——对照实验的全部价值押在这个等级上
三条独立通道互证	四边形占比只有解原生文件才看得到 → 自写 FBX 解析器 + Blender mesh.polygons + 平台读数, 三条对到一个不差
口径学	算子耗时 vs 端到端 (16s vs 31s, 差近一倍, 混用会把工期估少一半); 量出来的写 GiB、算出来的写 GB, 看单位就知道这个数是量的还是算的
自己的文档也是待核实来源	README 里"927 万面"躺了两天, 同节下一行写的是"927K"——差十倍; 为此全项目重测, 发现 29 件资产根本没有超 10 万面的, "必须重拓扑"整句话是错的
宁可缺数不凑数	14 天免费钱包不领 ("不能为了在书里凑一个数把它烧掉"); B 件生成耗时取不回来就空着那格

🔗 与 vault [[证据优先质检ProofOverClaims]] (PaperRoute 补强"证据可定位到帧") 同源且更进一步: 证据要有 **口径标签和独立通道数**。亦呼应 [[质检Gate与自我纠错循环]]、[[开源验货三查]] (本书还顺手示范了验货: 接口返回内部名 Nexus-v2.0-20260801 与文档别名 tripo-p2 的对账)。

收获 这本书给我什么: 能力清单

能力	怎么用
① 图生 3D 全流程认知	设计图 → Tripo → GLB → 清洗 → 引擎；要做 3D 场景时直接照 §17 四步流水线走
② 积分经济学	生成选 2K 导出 1k、按用途分档面数（建筑 15000/人车 8000/街具 6000）、Agent 批量前先验 ArrowRight
③ 11 个零依赖工具脚本	repo 的 scripts/ 直接可用：measure_glb.py（无引擎量 GLB 真包围盒）、E1/E2 减面流水线、glb_downscale、fbx2glb
④ Three.js 接资产检查表	§12 末尾 17 行检查表（环境贴图/相对路径/http/SkeletonUtils/normalized...），接任何第三方 GLB 前过一遍
⑤ 性能测量协议	GPU timer query + 串行 + 固定采样 + “量不出回归”的诚实结论
⑥ 验证文化模板	SHA-256 锁输入 / 多通道互证 / 口径标注 / 空格不抄数——写实验报告的标杆

对 AI Agent 用户的特别提示：本书目标读者就是“让 Codex/Claude Code 批量跑表单”的人——响应式输入框陷阱、浏览器自动化停在系统“另存为”弹窗（解法：API 直链绕开导出按钮）、失败任务不扣分（退出码 6 自动退款），都是 Agent 批量生产 3D 资产会撞的真实墙。

实验 两个离线实验（exercise\全真跑通）

实验一：零依赖手写 GLB + 减面极限复现 (验 \$04 E1)

步骤1 怎么操作： `python make_glb.py .` (零依赖, 照 paperroute 手写 GLB 先例) 造两个网格: 光滑 UV 球 (连续曲面) 和格构塔 (细钢梁结构), 都挂 `metalness=1` 铬材质。**目的：**拿到完全可控的测试资产。**实现了：** `sphere_smooth.glb` (720 三角) + `tower_lattice.glb` (1152 三角), 与笔算一致 (球 $2 \times 24 \times 16 - 48 = 720$; 塔 8 层 $\times 144 = 1152$)。**验证：**

```
sphere_smooth.glb  bytes=14560  tris=720   verts=425  (indices.count/3=720)
tower_lattice.glb  bytes=23940  tris=1152  verts=768  (indices.count/3=1152)
```

步骤2 怎么操作：双通道对账——自写 `count_faces` (\$04 口径 `indices.count/3`) + 书 repo 的 `measure_glb.py`。**验证：**两通道顶点数一致 (425/768), 包围盒与构造值吻合 (球 $2 \times 2 \times 2$ 、塔 $1.0 \times 1.0 \times 3.0$)。**意外收获：**塔按 Z-up 建, `measure_glb.py` 按 glTF Y-up 惯例把 Y 向 1.0008 当“高度”读出——亲手撞上书里“资产轴向/朝向不能想当然”的坑。

步骤3 怎么操作： `npm i @gltf-transform/cli` (4.5.1 装成本地依赖, 印证书里“别每次 `npx`”的建议), 对两个网格各跑 `simplify`, `ratio 0.5` $\rightarrow$ `0.02`。**验证 (真实输出)：**

```
== sphere_smooth ==  ratio=0.5 $\rightarrow$ 359  0.2 $\rightarrow$ 144  0.1 $\rightarrow$ 71  0.05 $\rightarrow$ 35  0.02 $\rightarrow$ 42 (触底反弹=到底了)
== tower_lattice ==  ratio=0.5 $\rightarrow$ 576  0.2 $\rightarrow$ 336  0.1 $\rightarrow$ 336  0.05 $\rightarrow$ 336  0.02 $\rightarrow$ 336 (三档输出完全相同)
```

✅ **预期结果兑现：**光滑球面砍到 4.9% 还是个球; 格构塔 29.2% 触底、`ratio` 再小也不动——“减面下限由几何光滑度决定, 格构结构几乎砍不动”亲手复现。

步骤4 怎么操作： `npx gltf-transform optimize sphere_smooth.glb decim/sphere_opt.glb --compress quantize`。**验证：**14,560 $\rightarrow$ 8,472 字节 (-42%), 量化压缩不需要运行时解码器 (书里选 `quantize` 不选 `draco/meshopt` 的理由: 沙盒里 WASM 解码器可能加载失败)。

实验二：PBR 白底死黑 A/B 复现与修复（验 §12 坑一）

步骤1 怎么操作： `npm i three@0.160.0`，从包里精选 5 个文件拷进 vendor/（书里 vendor 法，版本钉死防混版报错）。**验证：** 5 文件就位，含“不是可选的” BufferGeometryUtils.js。

步骤2 怎么操作： 写 shell.js（书 §12 最小壳本地化）：白底场景 + 实验一的铬球，`?env=0/1` 开关切换是否挂 studioEnv() canvas 渐变环境贴图；`python -m http.server 8901` 起服务（书里红线：WebGL 页面必须走 http，file:// 会被 CORS 挡）。**目的：** A/B 隔离唯一变量=环境贴图。

步骤3 怎么操作： headless Edge（`--use-angle=swiftshader` 软渲染）对 4 个组合截图。**验证：** 本指南 Q6 的两张实拍图即截图原文——env=0 死黑、env=1 正常镀铬球。

步骤4 怎么操作： 用零依赖 Python（zlib+struct 解 PNG 反滤波）量化球中心区平均亮度。**验证（真实输出）：**

球体中心区 (250,150)-(390,350):
env OFF: 72.9 env ON : 206.7 → 2.8×

✅ "白底 + 高金属度 + 无环境贴图 = 死黑；补 canvas 渐变环境贴图（带暗部）即修复"完整复现。附带发现：铬材质格构塔在白底上近乎隐形（细梁反射白底）——死黑坑的镜像现象。

入库 知识互链与查重结果

动作	条目	理由
新建	[[图生3D资产管线]]	商业图生3D的产品化全流程（两代模式/积分经济/五道接入工序/近景AI远景程序化分层）；与开源研究版 [[LRM大型重建模型]] 互补不重复
新建	[[面数口径与减面下限]]	面数框数的对象随模式变、1+四边形占比系数、光滑度决定下限、绘制调用优先于三角面
新建	[[先量后优化性能法]]	GPU timer query/vsync 陷阱/串行测量/"量不出回归"——先测量后动手的性能纪律
补强	[[Three.js与场景图]]	追加：环境贴图与 PBR 死黑、SkeletonUtils.clone 骨骼复制、vendor 钉版本法
补强	[[证据优先质检 ProofOverClaims]]	追加：口径学（算子vs端到端、GiB vs GB）、SHA-256 锁输入、多通道互证、"自己的文档也是待核实来源"

互链网：[[glTF与GLB资产管线]]（产物格式与 normalized 坑）、[[程序化建模]]（远景层）、[[混合模型分工流水线]]（设计图→提示词→GLB 的工序链）、[[机制流与资产流隔离]]（近景资产流/机制代码流）、[[Triplane三平面表示]]、[[点图与前馈重建]]、[[内容型开源与双许可]]（本书 CC BY-NC-SA 4.0 + Three.js MIT + OSM ODbL 三层许可划分）、[[MCP协议]]（tripo mcp 路线）、[[第二大脑LLMWiki]]（vault 本身）。

一句话总结：这是一本"把踩过的每个坑都变成可核查数字"的开源手册——3D 部分教你从一张设计图到一座能开车逛的城；方法论部分教你什么叫"面数是数出来的，不是想出来的"。它的验证纪律（哈希锁输入、多通道互证、口径标注、空格不抄数）比任何一个 3D 技巧都更值得搬走。

指南生成：2026-10-03 · 拾遗任务 79c4087a · 实验 material: F:\reminder\3d_vibe_coding\exercise\ · 上游 repo: alchaincyf/3d-vibe-coding-handbook @ 272a90e (CC BY-NC-SA 4.0)